

PROJECT CHARTER

1. Descrizione del progetto

Il progetto "Cruise-IoT Monitor" consiste nella realizzazione di un sistema IoT per il monitoraggio ambientale delle cabine di una nave da crociera. Lo scopo principale è controllare in tempo reale i valori di temperatura e umidità attraverso un sensore fisico (DHT22) collegato a un'infrastruttura IoT composta da Raspberry Pi, protocollo MQTT, piattaforma ThingsBoard e dashboard web accessibile da browser.

Il sistema simula uno scenario realistico in cui una nave con 3 ponti e 9 cabine totali invia dati ambientali a una piattaforma centrale. I dati vengono raccolti automaticamente, trasmessi attraverso la rete e visualizzati in una dashboard moderna, consultabile anche da remoto tramite smartphone o tablet.

2. Obiettivi del progetto

2.1 Obiettivo principale

Sviluppare un sistema funzionante di monitoraggio ambientale capace di:

- Ricevere dati reali dal sensore DHT22 tramite script dc.py
- Trasmettere i dati al gateway IoT (script iotgwda.py) con cifratura
- Pubblicare i dati cifrati via protocollo MQTT sul broker HiveMQ
- Archiviare i dati nel file dbplatform.json tramite script archivia_iotp.py
- Visualizzare temperatura e umidità in tempo reale su dashboard ThingsBoard

2.2 Obiettivi secondari

- Apprendere la progettazione di sistemi IoT con tecnologie professionali reali
- Migliorare le capacità di organizzazione, pianificazione e documentazione del team
- Produrre documentazione tecnica completa (GPOI e TPSIT)
- Gestire il ciclo di vita del progetto dalla concezione alla consegna finale

3. Contesto Applicativo e Stakeholder

3.1 Contesto

Nelle moderne navi da crociera il monitoraggio ambientale è fondamentale per garantire il comfort dei passeggeri e la sicurezza dell'imbarcazione. Anomalie di temperatura o umidità possono segnalare guasti tecnici che richiedono intervento immediato. Questo progetto simula tale sistema automatizzando la raccolta dati e la loro visualizzazione centralizzata.

3.2 Stakeholder

Stakeholder	Ruolo	Interesse nel Progetto
-------------	-------	------------------------

Equipaggio / Tecnici	Utenti primari	Monitorare lo stato delle cabine e gestire gli allarmi in tempo reale
Compagnia Crocieristica	Committente (simulato)	Garantire sicurezza, comfort dei passeggeri e efficienza operativa
Passeggeri	Beneficiari indiretti	Usufruire di condizioni ambientali confortevoli e sicure in cabina
Docenti (GPOI/TPSIT)	Valutatori	Verificare competenze tecniche e gestionali del team

4. Team di Progetto

Nome e Cognome	Ruolo nel Team	Responsabilità Principali
Noemi Basaglia	Project Manager / DC	Coordinamento generale, documentazione GPOI, configurazione Docker Desktop e ThingsBoard, aggiunta dispositivi su ThingsBoard, configurazione e gestione allarmi
Luigi Zara	Gateway IoT / DA	Sviluppo dashboard ThingsBoard, sviluppo e configurazione del codice per l'invio dei dati alla piattaforma
Mattia Crepaldi	Networking	Gestione allarmi, sviluppo codice invio dati a ThingsBoard, integrazione e sistemazione dell'intera pipeline DC → DA → ThingsBoard
Lorenzo Malachin	Dashboard	Supporto sviluppo dashboard ThingsBoard, contributo alla documentazione GPOI del progetto

5. Matrice di Responsabilità (RACI)

Legenda: R = Responsible (esegue) A = Accountable (risponde del risultato) C = Consulted (consultato) I = Informed (informato)

Macro-Attività	Noemi B.	Luigi Z.	Mattia C.	Lorenzo M.
Docker Desktop e ThingsBoard setup	R	I	C	I
Aggiunta dispositivi su ThingsBoard	R	C	C	I

Configurazione e gestione allarmi	R	I	R	I
Dashboard ThingsBoard (sviluppo)	C	R	C	C
Codice invio dati a ThingsBoard	I	R	R	I
Integrazione pipeline DC → DA → TB	C	I	R	I
Documentazione GPOI	A	I	C	R

6. Scope del Progetto

6.1 In Scope

- Acquisizione dati di temperatura e umidità tramite sensore DHT22 su Raspberry Pi
- Comunicazione dati via protocollo MQTT con broker HiveMQ (cloud)
- Cifratura del payload prima della pubblicazione MQTT
- Archiviazione dati in formato JSON (dbplatform.json)
- Integrazione con piattaforma ThingsBoard (istanza locale via Docker)
- Dashboard con gauge real-time, grafici time-series 24h, stato connettività, allarmi
- Gestione dinamica delle cabine tramite Entity Alias (1 nave, 3 ponti, 9 cabine)
- Documentazione completa: Project Charter, WBS, Gantt, Piano Rischi, Test Book

6.2 Out of Scope

- Deployment su nave reale o infrastruttura fisica navale
- Integrazione con sistemi di bordo esistenti (sistemi SCADA, BMS navali)
- Gestione di più navi simultanee (il progetto simula una sola nave)
- Applicazione mobile nativa (la dashboard è accessibile da browser responsive)
- Sistemi di autenticazione avanzata o gestione multi-utente su ThingsBoard

7. Funzionamento del Sistema

Il sistema è composto da cinque componenti principali che collaborano in pipeline:

#	Componente	Funzione
1	Sensore DHT22	Rileva periodicamente temperatura (°C) e umidità relativa (%) nella cabina
2	Script dc.py	Legge i dati dal sensore via GPIO e li invia al gateway IoT locale

3	Script iotgwda.py	Gateway IoT: riceve i dati, li cifra e li pubblica via MQTT sul topic cruise/nave01/ponteX/cabinaY
4	Script archivia_iotp.py	Subscriber MQTT: riceve i messaggi, li decifra e li archivia in dbplatform.json
5	ThingsBoard + Dashboard	Piattaforma IoT (Docker) che visualizza dati real-time, gestisce allarmi e permette consultazione da remoto

8. Tecnologie Utilizzate

Tecnologia	Tipo	Utilizzo nel Progetto
Raspberry Pi	Hardware	Piattaforma di elaborazione principale; esegue tutti gli script Python
Sensore DHT22	Hardware	Acquisizione di temperatura (0-50°C, $\pm 0.5^\circ\text{C}$) e umidità (0-100%, $\pm 2-5\%$)
Python 3	Software	Linguaggio usato per tutti gli script: dc.py, iotgwda.py, archivia_iotp.py
MQTT (HiveMQ)	Protocollo	Trasmissione leggera e affidabile dei dati IoT; topic gerarchici per cabina
JSON	Formato dati	Struttura del payload e formato di archiviazione in dbplatform.json
Docker	Infrastruttura	Containerizzazione dell'istanza ThingsBoard per isolamento e portabilità
ThingsBoard	Piattaforma IoT	Gestione dashboard, telemetrie, Entity Alias, widget e sistema di allarmi
WiFi / Ethernet	Rete	Connettività tra Raspberry Pi, broker MQTT e piattaforma ThingsBoard

9. Milestone di Progetto

Milestone	Data target	Criterio di completamento
M1 – Project Charter approvato	Gio 23/04/2026	Documento firmato e consegnato al docente
M2 – Sistema MQTT funzionante	Mer 29/04/2026	Dati sensore visibili sul broker HiveMQ

M3 – Archiviazione JSON attiva	Gio 07/05/2026	dbplatform.json aggiornato correttamente
M4 – Prima dashboard attiva	Sab 09/05/2026	Widget gauge e time-series visibili su ThingsBoard
M5 – Consegna finale	Mer 13/05/2026	Documentazione completa e sistema demo funzionante

10.Budget

Piano dei costi con stima indicativa minima/media/massima. Hardware e software sono già disponibili o gratuiti; il costo effettivo sostenuto dal team è pari a 0 €.

Voce di costo	Min (€)	Media (€)	Max (€)	Note
Raspberry Pi 4 Model B (2GB)	35,00	45,00	55,00	Già disponibile – costo 0
Sensore DHT22	3,00	5,00	8,00	Già disponibile – costo 0
Broker MQTT HiveMQ (piano free)	0,00	0,00	0,00	Piano gratuito sufficiente
Docker Desktop (licenza personal)	0,00	0,00	0,00	Gratuito per uso educativo
ThingsBoard Community Edition	0,00	0,00	0,00	Open source, gratuito
Cavi GPIO / accessori	2,00	5,00	10,00	Jumper wire, breadboard
Risorse umane- Noemi Basaglia (tariffa 30€/ora)	—	—	631,00	Costo calcolato in base alla tariffa orario
Risorse umane- Luigi Zara (tariffa 30€/ora)	—	—	736,00	Costo calcolato in base alla tariffa orario
Risorse umane- Mattia Crepaldi (tariffa 30€/ora)	—	—	435,50	Costo calcolato in base alla tariffa orario
Risorse umane- Lorenzo Malachin (tariffa 30€/ora)	—	—	120,00	Costo calcolato in base alla tariffa orario
Costo calcolato in base alla tariffa orario	—	—	166,50	Costo calcolato in base alla tariffa orario

TOTALE RISORSE UMANE			2.089,00 €	Somma costi
TOTALE COMPLESSIVO	40,00€	55,00€	2.162,00 €	Hardware stimato + risorse umane

Il costo hardware reale è 0 € (materiale già presente a scuola; software open source o a piano gratuito). I costi delle risorse umane (2.089,00 €) sono calcolati automaticamente da ProjectLibre in base alle ore pianificate nel Gantt e alla tariffa standard di 30,00 €/ora assegnata a ciascun membro. Trattandosi di un progetto scolastico, tale importo rappresenta il costo figurativo del lavoro e non un esborso effettivo.

11. Organizzazione del progetto – Standard di qualità e criteri di misurazione

Integrazione della sezione 2 con i criteri di qualità e le procedure operative del team.

Standard di qualità dei deliverable

Deliverable	Criteri di accettazione	Modalità di verifica
Script Python (dc.py, iotgwda.py, archivia_iotp.py)	Esecuzione senza errori; dati trasmessi correttamente nella pipeline	Test funzionale: avvio script e verifica output su console e broker MQTT
Dashboard ThingsBoard	Gauge e grafici time-series aggiornati entro 5 secondi; allarmi attivi sulle soglie	Verifica visiva da browser; test superamento soglia temperatura/umidità
File dbplatform.json	Record aggiornati ad ogni ciclo; struttura JSON valida; nessuna corruzione	Ispezione con editor JSON; confronto timestamp con orario di acquisizione
Documentazione (Charter, WBS, Gantt...)	Documenti completi, coerenti tra loro e consegnati entro le milestone	Revisione docente GPOI; checklist interna prima della consegna

Procedure operative adottate dal team

- *Comunicazione interna: aggiornamenti via chat di gruppo; riunione di allineamento ad ogni giovedì (giornata più lunga).*
- *Gestione del codice: repository condiviso su GitHub; commit con messaggi descrittivi per ogni modifica.*
- *Controllo avanzamento: identificazione precoce di task in ritardo.*
- *Procedura di chiusura: test finale pipeline completa; compilazione Test Book; consegna documentazione al docente entro mercoledì 13 maggio 2026 (M5).*

12. Vincoli e rischi – Tabella di valutazione pesata

Tabella con pesi e valutazione pesata secondo il modello del libro. Scala 1–3 per probabilità e impatto; valutazione pesata = peso × valutazione.

ID	Rischio	Prob.	Impatto	Peso	Val.	Strategia di mitigazione
R01	Interruzione connessione di rete	Media (2)	Alto (3)	0.20	0.60	Retry automatico MQTT; test riconnessione automatica
R02	Malfunzionamento sensore DHT22	Bassa (1)	Alto (3)	0.20	0.60	Sensore di riserva; range check nel codice
R03	Perdita dati in dbplatform.json	Bassa (1)	Alto (3)	0.15	0.45	Backup giornaliero; append atomico
R04	Inaccessibilità ThingsBoard dall'esterno	Media (2)	Medio (2)	0.15	0.30	Verifica firewall/porte; documentare IP e porta
R05	Assenza di un membro del team	Bassa (1)	Medio (2)	0.15	0.30	Rotazione conoscenze: ogni membro conosce 2+ componenti
R06	Ritardi nello sviluppo	Media (2)	Medio (2)	0.15	0.30	Revisione settimanale Gantt; early warning su task critici
TOTALE RISCHIO PESATO					2.55	Livello di rischio complessivo: MEDIO

13. Risultati Attesi

Al termine del progetto il sistema dovrà soddisfare i seguenti criteri di accettazione:

- Il sensore DHT22 invia dati di temperatura e umidità senza interruzioni per almeno 30 minuti consecutivi
- I dati transitano correttamente attraverso l'intera pipeline: sensore → dc.py → iotgwda.py → MQTT → archivia_iotp.py → dbplatform.json
- La dashboard ThingsBoard mostra i valori aggiornati entro 5 secondi dall'acquisizione
- I grafici time-series mostrano correttamente lo storico delle ultime 24 ore
- Il sistema di allarmi si attiva automaticamente al superamento delle soglie configurate
- La dashboard è accessibile e fruibile da dispositivi mobile (smartphone/tablet)
- La documentazione prodotta (Project Charter, WBS, Gantt, Piano Rischi, Test Book) è completa e coerente

13. Vincoli e Assunzioni

13.1 Vincoli

- Durata del progetto: 21 giorni lavorativi (23 aprile – 13 maggio 2026)
- Il team è composto da 4 studenti con disponibilità limitata alle ore scolastiche

- Infrastruttura hardware limitata a un singolo Raspberry Pi e un sensore DHT22
- ThingsBoard deve essere eseguito in locale tramite Docker

13.2 Assunzioni

- La connessione di rete scolastica consente l'accesso al broker MQTT HiveMQ (cloud)
- Il Raspberry Pi rimane acceso e connesso per tutta la durata delle sessioni di test
- La licenza Community di ThingsBoard è sufficiente per le funzionalità richieste